

Incentive-Aware Machine Learning

Robustness, Fairness, Improvement & Causality

Chara Podimata

Senem Işık

Motivation

strategic adaptation

Machine Learning (ML) algorithms are increasingly being used for making critical decisions in various aspects of modern life.

e.g. hiring, loan approvals, college admissions, and probation rulings...



Motivation

strategic adaptation

Machine Learning (ML) algorithms are increasingly being used for making critical decisions in various aspects of modern life.

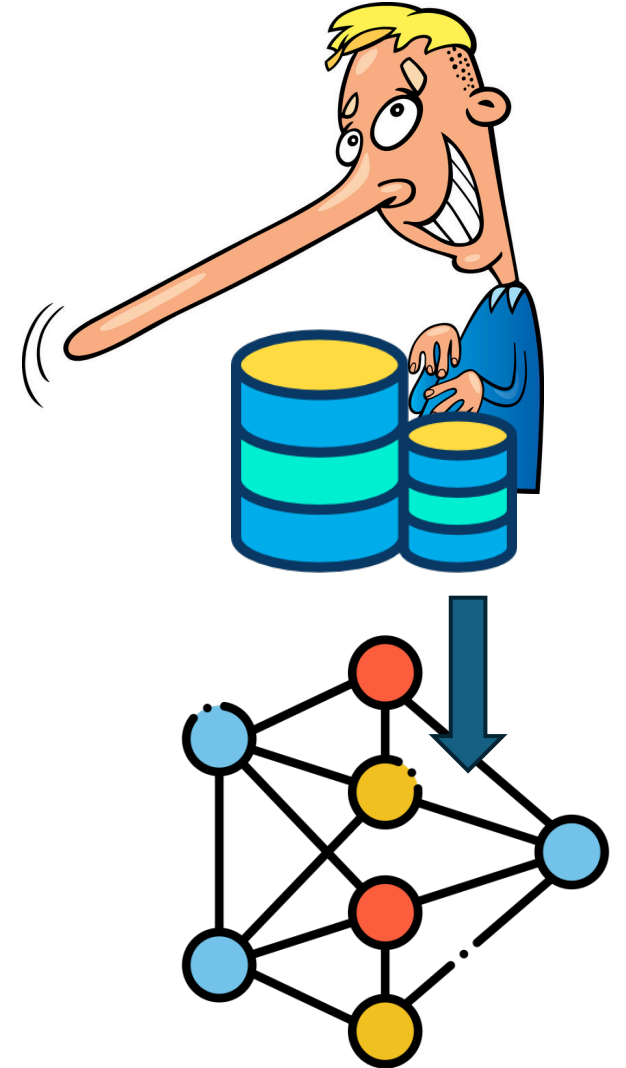
e.g. hiring, loan approvals, college admissions, and probation rulings...

Given the high stakes of these decisions, **individuals often have strong incentives to strategically modify the data** they provide to these algorithms.

eg. opening additional credit accounts or take other steps to improve their credit scores before applying for a loan, switch schools to boost their class rankings for college admissions...

Main Questions in **strategic classification** or **performative prediction**

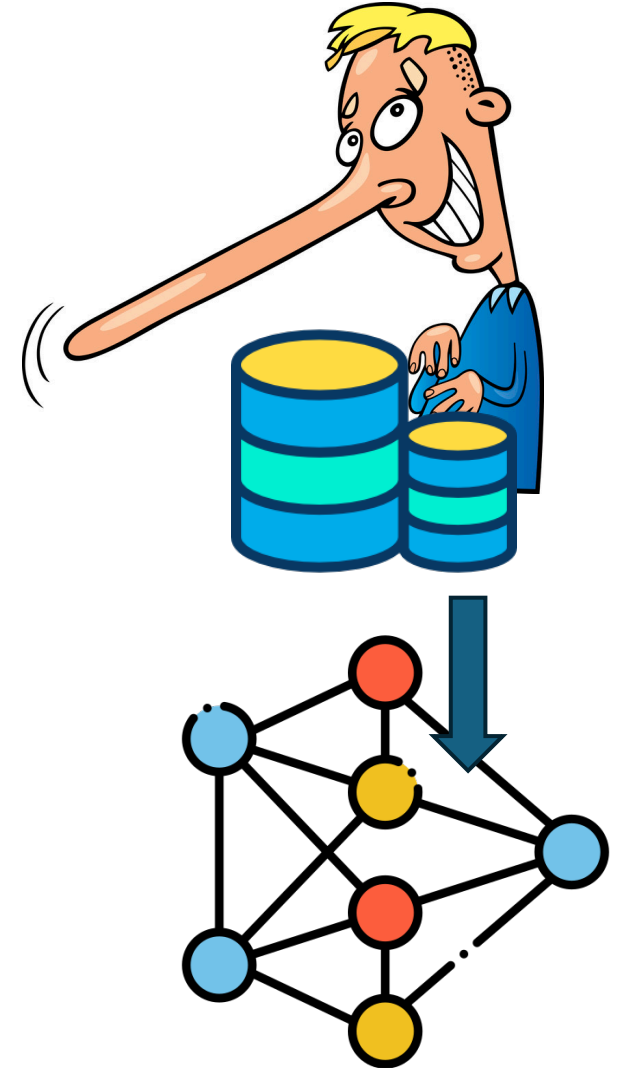
What should decision-makers do when
individuals are incentivized to alter the data
they provide to ML algorithms in pursuit of better outcomes?



Main Questions in **strategic classification** or **performative prediction**

What should decision-makers do when
individuals are incentivized to alter the data
they provide to ML algorithms in pursuit of better outcomes?

And even if the **learner manages to robustify (or calibrate)**
their algorithms to account for such behavior,
what are the societal implications?



Main Perspectives

(1) Robustness

- viewpoint of the decision-maker: *agents always attempt to “game” the decision rule*
- goal: *achieve optimality despite strategic adaptations by the agents*

Main Perspectives

(1) Robustness

- viewpoint of the decision-maker: *agents always attempt to “game” the decision rule*
- goal: *achieve optimality despite strategic adaptations by the agents*

(2) Fairness

- the downstream societal impacts of algorithms under varying assumptions about the agents' **capacity** to strategically adapt

Main Perspectives

(1) Robustness

- viewpoint of the decision-maker: *agents always attempt to “game” the decision rule*
- goal: *achieve optimality despite strategic adaptations by the agents*

(2) Fairness

- the downstream societal impacts of algorithms under varying assumptions about the agents' **capacity** to strategically adapt

(3) Improvement & Causality

- **not** all strategic adaptations are **harmful**: *sometimes agents' adaptations lead to genuine, fundamental improvements*

Main Perspectives

(1) Robustness

- viewpoint of the decision-maker: *agents always attempt to “game” the decision rule*
- goal: *achieve optimality despite strategic adaptations by the agents*

(2) Fairness

- the downstream societal impacts of algorithms under varying assumptions about the agents' **capacity** to strategically adapt

(3) Improvement & Causality

- **not** all strategic adaptations are **harmful**: *sometimes agents' adaptations lead to genuine, fundamental improvements*

Incentive-aware learning as a game

- between a **principal** (aka learner, decision-maker) and **agents**
- studied both in the **offline**
(i.e., where there is one decision that is made by the principal and then the interaction stops)

and **online** setting

(i.e., where there are sequential decisions)



Notation

- Let $\mathcal{X} \in \mathbb{R}^d$ be the *feature space*
- $\mathcal{Y} = \{0,1\}$ (respectively $\mathcal{Y} \subseteq [0,1]$ for linear regression) the *label* (respectively response) space
- Assume label (respectively response) is $y = h^*(x)$ where $h^*: \mathcal{X} \rightarrow \mathcal{Y}$ is called the *ground truth* function (which is not necessarily linear)
- $\mathcal{H}$ be the *concept class* where h^* belongs to
- $\ell: \mathcal{Y} \times \mathcal{Y} \rightarrow [0,1]$ be the principal's **loss function**:
 - 0 – 1 loss: $\ell(y, y') := 1\{\text{sign}(y \cdot y') = -1\}$
 - Logistic loss: $\ell(y, y') := \log(1 + e^{-y \cdot y'})$
 - Hinge loss: $\ell(y, y') := \max(0, 1 - y \cdot y')$

For the regression tasks, the most commonly used loss is some L_p norm

Offline Setting as *Stackelberg game*

- (1) Nature draws $x \sim \mathcal{D}$.
- (2) The principal ("leader")—without knowing x —commits to (and publicly announces) a decision-making rule $f: \mathcal{X} \rightarrow \mathcal{Y}$.

Offline Setting as *Stackelberg game*

- (1) Nature draws $x \sim \mathcal{D}$.
- (2) The principal ("leader")—without knowing x —commits to (and publicly announces) a decision-making rule $f: \mathcal{X} \rightarrow \mathcal{Y}$.
- (3) The agents ("follower) observe f and their point (x, y) .
- (4) Given f, x, y the agents choose $\hat{x} = \hat{x}(f; x, y) \in \mathcal{X}$ is the best-response of the agent (given pair (x, y)) to the principal's rule f .
- (5) The agent reports point $\hat{x}$ to the principal.

Offline Setting as *Stackelberg game*

- (1) Nature draws $x \sim \mathcal{D}$.
- (2) The principal ("leader")—without knowing x —commits to (and publicly announces) a decision-making rule $f: \mathcal{X} \rightarrow \mathcal{Y}$.
- (3) The agents ("follower) observe f and their point (x, y) .
agent obtains a better standing with regards to f
- (4) Given f, x, y the agents choose $\hat{x} = \hat{x}(f; x, y) \in \mathcal{X}$ is the best-response of the agent (given pair (x, y)) to the principal's rule f .
- (5) The agent reports point $\hat{x}$ to the principal.

Principal's goal

- (1) Nature draws $x \sim \mathcal{D}$.
- (2) The principal ("leader")— without knowing x —commits to (and publicly announces) a decision-making rule $f: \mathcal{X} \rightarrow \mathcal{Y}$.
- (3) The agents ("follower) observe f and their point (x, y) .
- (4) Given f, x, y the agents choose $\hat{x} = \hat{x}(f; x, y) \in \mathcal{X}$ is the best-response of the agent (given pair (x, y)) to the principal's rule f .
agent obtains a better standing with regards to f
- (5) The agent reports point $\hat{x}$ to the principal.

Find a function $f^* \in \mathcal{F}$ (where $\mathcal{F}: \mathcal{X} \rightarrow \mathcal{Y}$ is a hypothesis class over which we are searching) such that

$$f^* = \arg \min_{f \in \mathcal{F}} \mathbb{E}_{x \in \mathcal{D}} [\ell(h^*(x), f(\hat{x}(f)))]$$

Online Setting

The interaction occurs over $t \in [T]$ rounds

- (1) Nature draws $x_t \sim \mathcal{D}$.
- (2) The principal—without knowing x_t —commits to (and publicly announces) a decision-making rule $f_t: \mathcal{X} \rightarrow \mathcal{Y}$.
- (3) The agents observe f_t and their point (x_t, y_t) .
- (4) Given f_t, x_t, y_t the agents choose $\hat{x}_t = \hat{x}_t(f_t; x_t, y_t) \in \mathcal{X}$ is the best-response of the agent (given pair (x_t, y_t)) to the principal's rule f_t .
- (5) The agent reports point $\hat{x}_t$ to the principal.

Online Setting

The interaction occurs over $t \in [T]$ rounds

- (1) Nature draws $x_t \sim \mathcal{D}$.
 - (2) The principal—without knowing x_t —commits to (and publicly announces) a decision-making rule $f_t: \mathcal{X} \rightarrow \mathcal{Y}$.
 - (3) The agents observe f_t and their point (x_t, y_t) .
 - (4) Given f_t, x_t, y_t the agents choose $\hat{x}_t = \hat{x}_t(f_t; x_t, y_t) \in \mathcal{X}$ is the best-response of the agent (given pair (x_t, y_t)) to the principal's rule f_t .
 - (5) The agent reports point $\hat{x}_t$ to the principal.
- The sequence $\{x_t\}_{t \in [T]}$ that nature chooses can be *adversarial*.
 - $\mathcal{F}$ can be a general class of functions but we will focus on *linear* ones.
 - We assume that when the agent strategically adapts to a rule f_t , they cannot influence their y_t
(i.e., y_t remains the same both for x_t and for the misreport $\hat{x}_t$)

Principal's goal

The interaction occurs over $t \in [T]$ rounds

- (1) Nature draws $x_t \sim \mathcal{D}$.
- (2) The principal—without knowing x_t —commits to (and publicly announces) a decision-making rule $f_t: \mathcal{X} \rightarrow \mathcal{Y}$.
- (3) The agents observe f_t and their point (x_t, y_t) .
- (4) Given f_t, x_t, y_t the agents choose $\hat{x}_t = \hat{x}_t(f_t; x_t, y_t) \in \mathcal{X}$ is the best-response of the agent (given pair (x_t, y_t)) to the principal's rule f_t .
- (5) The agent reports point $\hat{x}_t$ to the principal.

minimize **Stackelberg regret**:

$$Reg(T) := \sum_{t \in [T]} \ell \left(f_t(\hat{x}_t(f_t)), h^*(x_t) \right) - \min_{f^{\text{OPT}} \in \mathcal{F}} \sum_{t \in [T]} \ell \left(f^{\text{OPT}}(\hat{x}_t(f^{\text{OPT}})), h^*(x_t) \right)$$

(we are comparing the algorithm's performance to the best *fixed* rule had you given the agents the opportunity to best respond)

Learning Strategy-Aware Linear Classifiers

Yiling Chen, Yang Liu, Chara Podimata

Senem Işık

Interaction Protocol

Running example: Each agent has an email that is either a spam or a non-spam. Given a classifier, the agent can alter his email to a certain degree to bypass the email.

- (1) Nature adversarially selects feature vector $x_t \in \mathcal{X} \subseteq ([0,1]^d, 1)$ [the agent's original email]
- (2) The learner, not knowing x_t , commits action $f_t \subseteq [-1, +1]^{d+1}$ [hyperplanes i.e. linear classifiers]
- (3) The agents observes f_t (and their classification $y_t = \pm 1$) [spam?]
- (4) Given f_t, x_t, y_t the agent reports $\hat{x}_t(f_t; x_t, y_t) \in \mathcal{X}$. [the agent's original email]
- (5) The learner observes $(\hat{x}_t := \hat{x}_t(f_t; x_t, y_t), y_t)$ where $\hat{y}_t \in \mathcal{Y}$ and incurs a binary loss
$$\ell(f_t, \hat{x}_t, \hat{y}_t) := 1\{\text{sign}(\hat{y}_t \times (f_t \cdot \hat{x}_t,)) = -1\}$$

(recall $\hat{y}_t = y_t$)

⚠ This metric is meant to capture the loss incurred by manipulation and not accuracy. eg. if the email was classified as non-spam originally (while being spam) and still is, no extra loss.

Agents' Behavior: δ -Bounded Myopically Rational

Manipulation is costly.

$$\hat{x}_t(f_t; x_t, y_t) = \arg \max_{\|\hat{x} - x_t\| \leq \delta} u_t(f_t, \hat{x})$$

where u_t is the agent's utility function, which is *unknown to the learner*, but δ is known.

In words, in choosing what to report,

- the agents are myopic (i.e., focus only on the current round t),
- rational (i.e., maximize their utility),
- and bounded (i.e., misreport in a ball of radius δ around x_t)

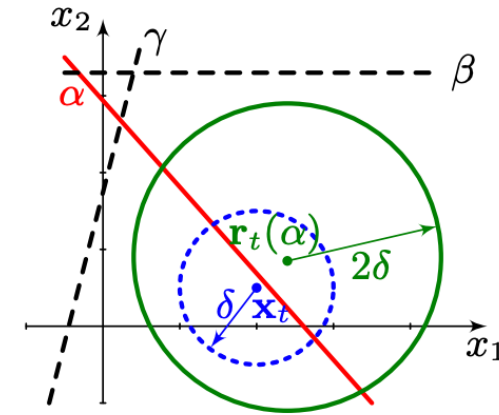


Figure 2: Agent's action space with axes x_1, x_2 corresponding to features. $\mathbf{x}_t$ is the agent's true feature vector and $\mathbf{r}_t(\alpha)$ his misreport against α . Actions α, β, γ comprise the learner's action set.

Agents' Behavior: δ -Bounded Myopically Rational

⚠ Manipulation does **not** make the spam email non-spam $\rightarrow y_t = \hat{y}_t$.

⚠ Unlike previous work, assumptions on utility is very general. For example, this model includes any utility defined as

$$\delta' 1\{\text{sgn}(f_t \cdot \hat{x}_t) = 1\} - \|x_t - \hat{x}_t\|_2$$

for receiving value of δ' if the label is non-spam and incurring a cost (i.e., time/resources spent for altering the original x_t) that is a metric. Here, $\delta \leq \delta'$.

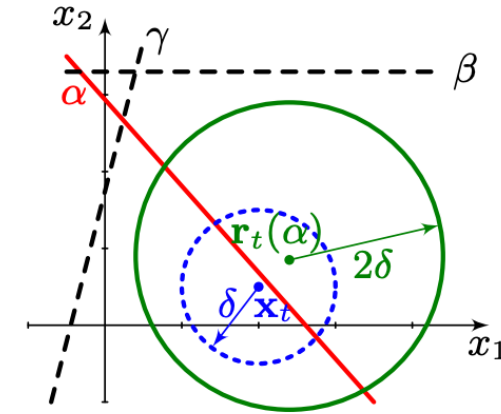


Figure 2: Agent's action space with axes x_1, x_2 corresponding to features. $\mathbf{x}_t$ is the agent's true feature vector and $\mathbf{r}_t(\alpha)$ his misreport against α . Actions α, β, γ comprise the learner's action set.

Stackelberg versus External Regret

Stackelberg regret:

$$Reg(T) := \sum_{t \in [T]} \ell(f_t(\hat{x}_t(f_t)), y_t) - \min_{f^{\text{OPT}} \in \mathcal{F}} \sum_{t \in [T]} \ell(y_t, f^{\text{OPT}}(\hat{x}_t(\textcolor{blue}{f}^{\text{OPT}})), y_t)$$

(we are comparing the algorithm's performance to the best *fixed* linear classifier in hindsight had the agent given the agents the opportunity to best respond)

External regret:

$$Reg(T) := \sum_{t \in [T]} \ell(y_t, f_t(\hat{x}_t(f_t)), y_t) - \min_{f^{\text{OPT}} \in \mathcal{F}} \sum_{t \in [T]} \ell(y_t, f^{\text{OPT}}(\hat{x}_t(\textcolor{blue}{f}_t)), y_t)$$

(we are comparing the algorithm's performance to the best *fixed* linear classifier in hindsight had the agent **not** given the agents the opportunity to best respond)

Main result I: strongly incompatibility

Theorem: There exists a repeated strategic classification setting against a δ -BMR agent, where every action sequence from the learner with **sublinear external regret** incurs **linear Stackelberg regret**, and every action sequence for the learner with **sublinear Stackelberg regret** incurs **linear external regret**.

Proof.

Main result I

Theorem: There exists a repeated strategic classification setting against a δ -BMR agent, where every action sequence from the learner with sublinear external regret incurs linear Stackelberg regret, and every action sequence for the learner with sublinear Stackelberg regret incurs linear external regret.

Proof.

$$p^1 = 0.05, p^2 = 0.15, p^3 = 0.05, p^4 = 0.75.$$

$$\begin{aligned} \mathbb{E}[\ell(h, \hat{x}_t(h), \hat{y}_t)] &= 0.2, \mathbb{E}[\ell(h', \hat{x}_t(h'), \hat{y}_t)] = 0.25, \\ \mathbb{E}[\ell(h, \hat{x}_t(h'), \hat{y}_t)] &= 0.9, \mathbb{E}[\ell(h', \hat{x}_t(h), \hat{y}_t)] = 0.05 \end{aligned}$$

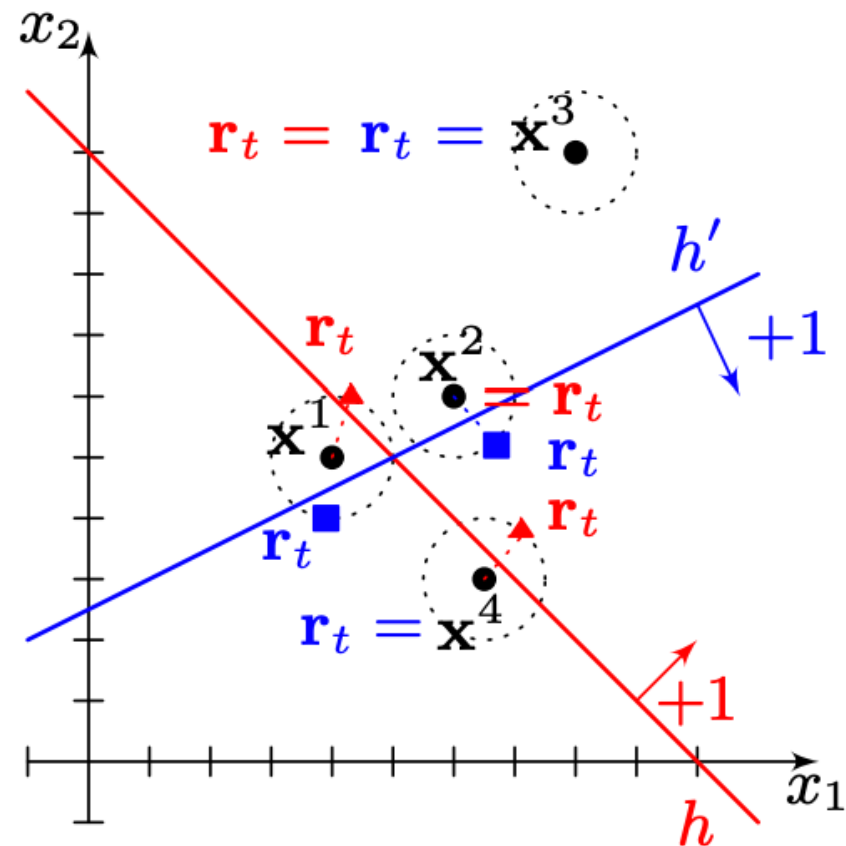


Figure 1: Black dots denote true feature vectors. Axes x_1, x_2 correspond to the two features. Dotted circles correspond to the δ -bounded interval inside which agents can misreport. Blue squares correspond to misreports against action h' and red triangles to misreports against action h .

Main result I

Theorem: There exists a repeated strategic classification setting against a δ -BMR agent, where every action sequence from the learner with sublinear external regret incurs linear Stackelberg regret, and every action sequence for the learner with sublinear Stackelberg regret incurs linear external regret.

Proof.

$$\mathbb{E}[\ell(h, \hat{x}_t(h), \hat{y}_t)] = 0.2, \mathbb{E}[\ell(h', \hat{x}_t(h'), \hat{y}_t)] = 0.25,$$

$$\mathbb{E}[\ell(h, \hat{x}_t(h'), \hat{y}_t)] = 0.9, \mathbb{E}[\ell(h', \hat{x}_t(h), \hat{y}_t)] = 0.05$$

→ For sublinear Stackelberg regret, one must do h for $T-o(T)$ iterations.

$$\mathbb{E}[\ell(h, \hat{x}_t(h), \hat{y}_t)] = 0.2, \mathbb{E}[\ell(h', \hat{x}_t(h'), \hat{y}_t)] = 0.25,$$

$$\mathbb{E}[\ell(h, \hat{x}_t(h'), \hat{y}_t)] = 0.9, \mathbb{E}[\ell(h', \hat{x}_t(h), \hat{y}_t)] = 0.05$$

→ This gives at least $0.15T$ external regret (linear)

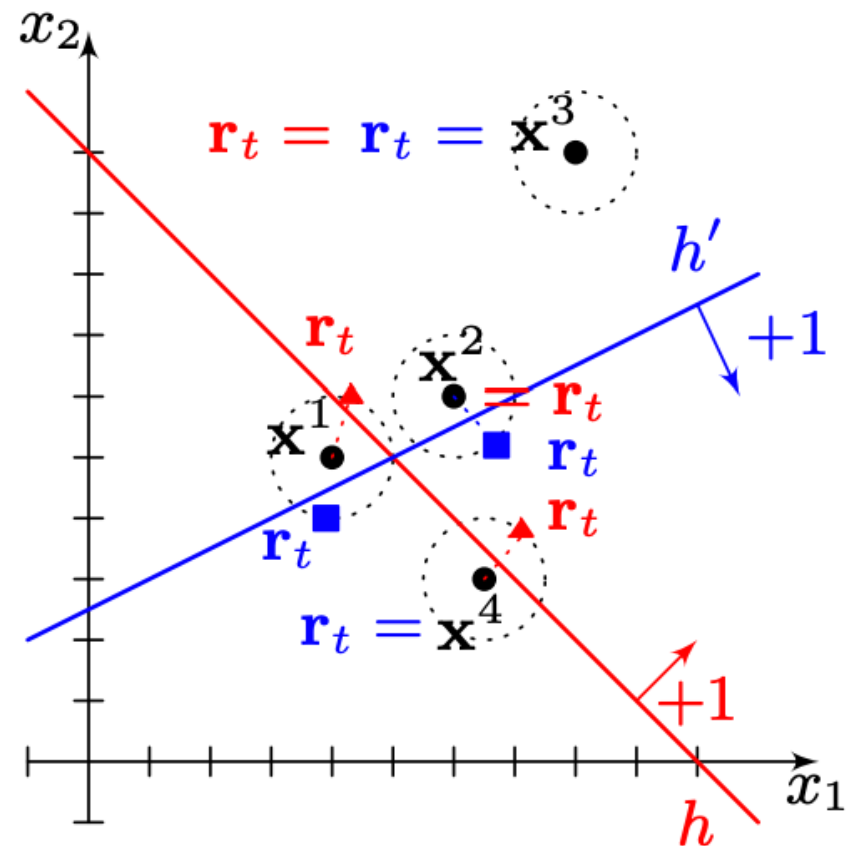


Figure 1: Black dots denote true feature vectors. Axes x_1, x_2 correspond to the two features. Dotted circles correspond to the δ -bounded interval inside which agents can misreport. Blue squares correspond to misreports against action h' and red triangles to misreports against action h .

Main result I

Theorem: There exists a repeated strategic classification setting against a δ -BMR agent, where every action sequence from the learner with sublinear external regret incurs linear Stackelberg regret, and every action sequence for the learner with sublinear Stackelberg regret incurs linear external regret.

Proof.

$$\mathbb{E}[\ell(h, \hat{x}_t(h), \hat{y}_t)] = 0.2, \mathbb{E}[\ell(h', \hat{x}_t(h'), \hat{y}_t)] = 0.25,$$

$$\mathbb{E}[\ell(h, \hat{x}_t(h'), \hat{y}_t)] = 0.9, \mathbb{E}[\ell(h', \hat{x}_t(h), \hat{y}_t)] = 0.05$$

→ For sublinear external regret, one must do h' for $T-o(T)$ iterations.

$$\mathbb{E}[\ell(h, \hat{x}_t(h), \hat{y}_t)] = 0.2, \mathbb{E}[\ell(h', \hat{x}_t(h'), \hat{y}_t)] = 0.25,$$

$$\mathbb{E}[\ell(h, \hat{x}_t(h'), \hat{y}_t)] = 0.9, \mathbb{E}[\ell(h', \hat{x}_t(h), \hat{y}_t)] = 0.05$$

→ This gives at least $0.05T$ Stackelberg regret (linear)

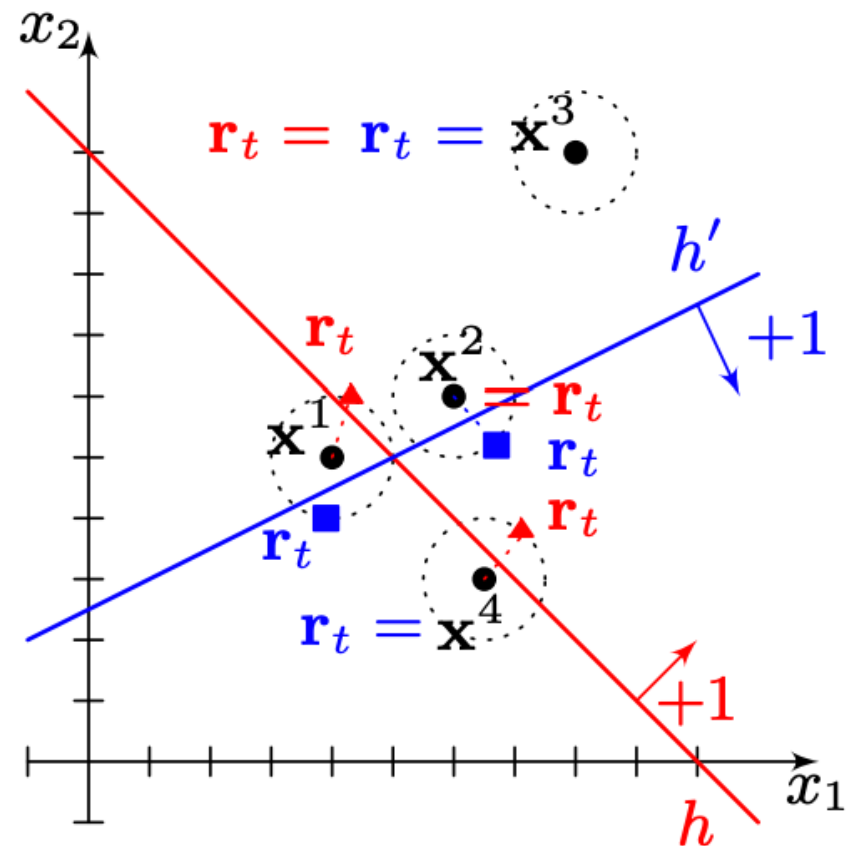


Figure 1: Black dots denote true feature vectors. Axes x_1, x_2 correspond to the two features. Dotted circles correspond to the δ -bounded interval inside which agents can misreport. Blue squares correspond to misreports against action h' and red triangles to misreports against action h .

Main result II

Inferring $\ell(y_t, f_t(\hat{x}_t(f_t)), y_t)$ without observing x_t

⚠ The agent observes $\hat{y}_t$ after each $t \in [T]$ to calculate its loss and knows δ .

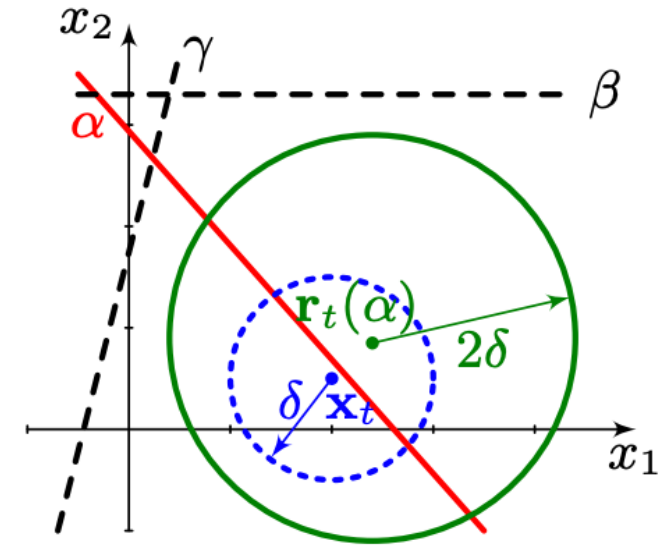


Figure 2: Agent's action space with axes x_1, x_2 corresponding to features. $\mathbf{x}_t$ is the agent's true feature vector and $\mathbf{r}_t(\alpha)$ his misreport against α . Actions α, β, γ comprise the learner's action set.

The Grinder Algorithm

Inferring $\ell(y_t, f_t(\hat{x}_t(f_t)), y_t)$ without observing x_t

⚠ The agent observes $\hat{y}_t$ after each $t \in [T]$ to calculate its loss and knows δ .

Then, if the augmented 2δ ball (green solid line) is in one of half-spaces of an action f_t (e.g. β, γ), then the learner can infer her loss for

$$\ell(f_t, \hat{x}_t(f_t), \hat{y}_t) := 1\{\text{sign}(\hat{y}_t \times (f_t \cdot \hat{x}_t(f_t))) = -1\}$$

This happens if and only if

$$\frac{|f_t \cdot \hat{x}_t(f_t)|}{\|h\|_2} \leq 2\delta \leftrightarrow |f_t \cdot \hat{x}_t(f_t)| \leq 4\sqrt{d}\delta$$

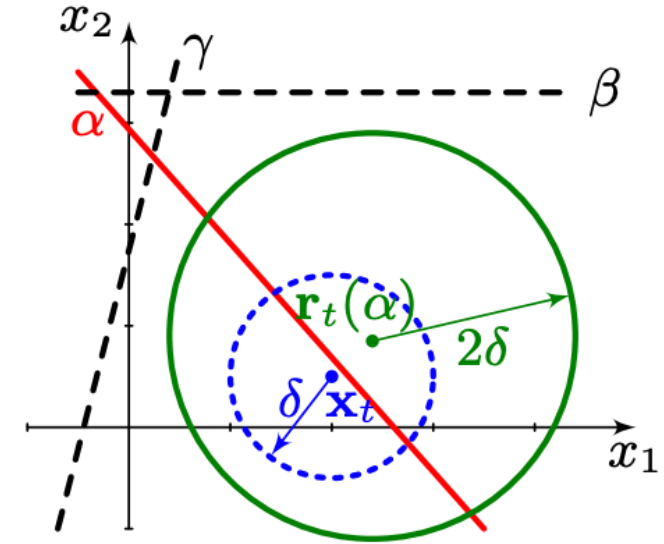


Figure 2: Agent's action space with axes x_1, x_2 corresponding to features. $\mathbf{x}_t$ is the agent's true feature vector and $\mathbf{r}_t(\alpha)$ his misreport against α . Actions α, β, γ comprise the learner's action set.

Dual Space $\{\mathcal{A}\}$ of the Learner

Given $\hat{x}_t$, define the upper polytope,

$$\beta_t^u(\hat{x}_t(f_t)) = \{\gamma \mid \gamma \cdot \hat{x}_t(f_t) \geq 4\sqrt{d}\delta\},$$

containing actions such that assigned label is +1, and the lower polytope:

$$\beta_t^l(\hat{x}_t(f_t)) = \{\gamma \mid \gamma \cdot \hat{x}_t(f_t) \leq -4\sqrt{d}\delta\}$$

containing actions such that assigned label is -1.

This partitions the dual space of the learner into 3 regions, and at each round the estimated loss within the two extreme polytope is constant ± 1 .

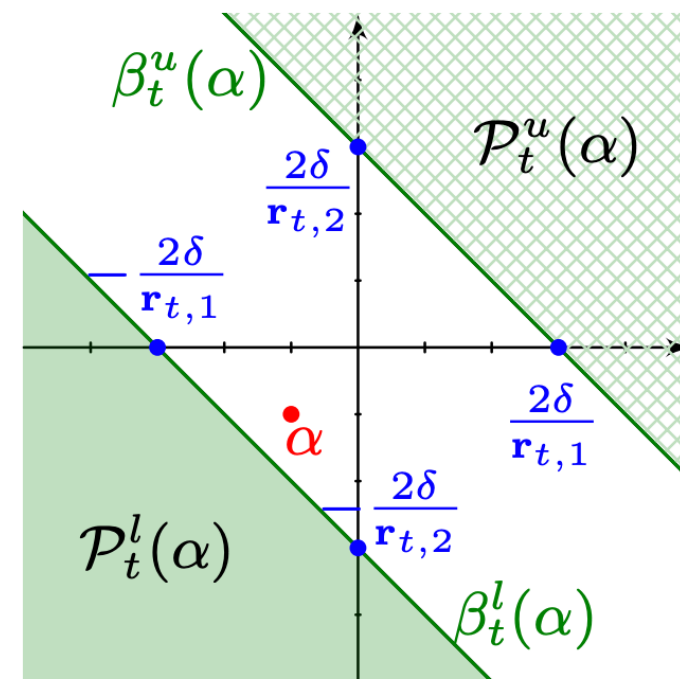


Figure 3: Polytope partitioning for $d = 2$. $\mathbf{r}_{t,1}, \mathbf{r}_{t,2}$ correspond to the x_1 and x_2 coordinates of $\mathbf{r}_t(\alpha)$.

The Grinder Algorithm

Algorithm 2: GRINDER Algorithm for Strategic Classification

```

1 Initialize polytopes' set:  $\mathcal{P}_0 = \{\mathcal{A}\}$ .
2 Initialize polytope weights  $w_1(p) = \lambda(p), p \in \mathcal{P}_0$ .
3 Tune learning and exploration rates  $\eta = \gamma \leq 1/2$ , as specified in the analysis.
4 for  $t \leftarrow 1$  to  $T$  do
5     Compute  $\forall p \in \mathcal{P}_t : \pi_t(p) = (1 - \gamma)q_t(p) + \gamma \frac{\lambda(p)}{\lambda(\mathcal{A})}$ . // distribution over polytopes
6     /* Two-stage sampling: first, polytope, second, draw action from within. */
7     Select polytope  $p_t \sim \pi_t$  from which you draw action  $\alpha_t \sim \text{Unif}(p_t)$  and commit to  $\alpha_t$ .
8     Observe the agent's response  $(\mathbf{r}_t(\alpha_t), y_t)$  to committed  $\alpha_t$ .
9     /* Space partitioning into smaller polytopes.  $\mathcal{P}_t$ : current polytopes set. */
10    Define a new set of polytopes  $\mathcal{P}_{t+1} = \mathcal{P}_{t+1}^u(\alpha_t) \cup \mathcal{P}_{t+1}^m(\alpha_t) \cup \mathcal{P}_{t+1}^l(\alpha_t)$ , where:
11    for each polytope  $p \in \mathcal{P}_t$  do
12        Add in  $\mathcal{P}_{t+1}^u(\alpha_t)$  the non-empty intersection  $p \cap H^+(\beta_t^u(\alpha_t))$  // upper polytopes set
13        Add in  $\mathcal{P}_{t+1}^l(\alpha_t)$  the non-empty intersection  $p \cap H^-(\beta_t^l(\alpha_t))$  // lower polytopes set
14        Add in  $\mathcal{P}_{t+1}^m(\alpha_t)$  the non-empty remainder of  $p$ . // middle polytopes set
15    Compute  $\hat{\ell}(\alpha_t, \mathbf{r}_t(\alpha_t), y_t) = \frac{\ell(\alpha_t, \mathbf{r}_t(\alpha_t), y_t)}{\mathbb{P}_t^{\text{in}}[\alpha_t]}$ . // loss estimator for chosen action
16    for each polytope  $p \in \mathcal{P}_{t+1}$  do
17        /* upper and lower polytopes get full information */
18        Compute  $\hat{\ell}(p, \mathbf{r}_t(p), y_t) = \frac{\ell(p, \mathbf{r}_t(p), y_t) \cdot \mathbb{1}\{p \subseteq \mathcal{P}_{t+1}^u(\alpha_t) \cup \mathcal{P}_{t+1}^l(\alpha_t)\}}{\mathbb{P}_t^{\text{in}}[p]}$ .
19        /* weight scaling with the Lebesgue measure of the polytope */
20        Update  $w_{t+1}(p) = \lambda(p) \exp\left(-\eta \sum_{\tau=1}^t \hat{\ell}(p, \mathbf{r}_\tau(p), y_\tau)\right)$ ,  $q_{t+1}(p) = \frac{w_{t+1}(p)}{\sum_{p' \in \mathcal{P}_{t+1}} w_{t+1}(p')}$ .

```

Let's partition the action space $\{\mathcal{A}\}$ recursively!

```
6 | Select polytope  $p_t \sim \pi_t$  from which you draw action  $\alpha_t \sim \text{Unif}(p_t)$  and commit to  $\alpha_t$ .
7 | Observe the agent's response  $(\mathbf{r}_t(\alpha_t), y_t)$  to committed  $\alpha_t$ .
   | /* Space partitioning into smaller polytopes.  $\mathcal{P}_t$ : current polytopes set. */
8 | Define a new set of polytopes  $\mathcal{P}_{t+1} = \mathcal{P}_{t+1}^u(\alpha_t) \cup \mathcal{P}_{t+1}^m(\alpha_t) \cup \mathcal{P}_{t+1}^l(\alpha_t)$ , where:
9 | for each polytope  $p \in \mathcal{P}_t$  do
10 | |   Add in  $\mathcal{P}_{t+1}^u(\alpha_t)$  the non-empty intersection  $p \cap H^+(\beta_t^u(\alpha_t))$            // upper polytopes set
11 | |   Add in  $\mathcal{P}_{t+1}^l(\alpha_t)$  the non-empty intersection  $p \cap H^-(\beta_t^l(\alpha_t))$            // lower polytopes set
12 | |   Add in  $\mathcal{P}_{t+1}^m(\alpha_t)$  the non-empty remainder of  $p$ .                               // middle polytopes set
13 | Compute  $\hat{\ell}(\alpha_t, \mathbf{r}_t(\alpha_t), y_t) = \frac{\ell(\alpha_t, \mathbf{r}_t(\alpha_t), y_t)}{\mathbb{P}_t^{\text{in}}[\alpha_t]}$ .           // loss estimator for chosen action
```

Estimate and update the loss for each partition!

```
13 | Compute  $\hat{\ell}(\alpha_t, \mathbf{r}_t(\alpha_t), y_t) = \frac{\ell(\alpha_t, \mathbf{r}_t(\alpha_t), y_t)}{\mathbb{P}_t^{\text{in}}[\alpha_t]}$ . // loss estimator for chosen action
14 | for each polytope  $p \in \mathcal{P}_{t+1}$  do
    | /* upper and lower polytopes get full information */
15 |     Compute  $\hat{\ell}(p, \mathbf{r}_t(p), y_t) = \frac{\ell(p, \mathbf{r}_t(p), y_t) \cdot \mathbb{1}\{p \subseteq \mathcal{P}_{t+1}^u(\alpha_t) \cup \mathcal{P}_{t+1}^l(\alpha_t)\}}{\mathbb{P}_t^{\text{in}}[p]}$ .
    | /* weight scaling with the Lebesgue measure of the polytope */
```

Update the weights and sample in two stages!

Algorithm 2: GRINDER Algorithm for Strategic Classification

- 1 Initialize polytopes' set: $\mathcal{P}_0 = \{\mathcal{A}\}$.
- 2 Initialize polytope weights $w_1(p) = \lambda(p), p \in \mathcal{P}_0$.
- 3 Tune learning and exploration rates $\eta = \gamma \leq 1/2$, as specified in the analysis.
- 4 **for** $t \leftarrow 1$ **to** T **do**
- 5 Compute $\forall p \in \mathcal{P}_t : \pi_t(p) = (1 - \gamma)q_t(p) + \gamma \frac{\lambda(p)}{\lambda(\mathcal{A})}$. // distribution over polytopes
- 6 /* Two-stage sampling: first, polytope, second, draw action from within. */
Select polytope $p_t \sim \pi_t$ from which you draw action $\alpha_t \sim \text{Unif}(p_t)$ and commit to α_t .

16

$\leftarrow t \leftarrow 1$

/* weight scaling with the Lebesgue measure of the polytope*/

Update $w_{t+1}(p) = \lambda(p) \exp \left(-\eta \sum_{\tau=1}^t \hat{\ell}(p, \mathbf{r}_\tau(p), y_\tau) \right)$, $q_{t+1}(p) = \frac{w_{t+1}(p)}{\sum_{p' \in \mathcal{P}_{t+1}} w_{t+1}(p')}$.

The Grinder Algorithm

Algorithm 2: GRINDER Algorithm for Strategic Classification

```
1 Initialize polytopes' set:  $\mathcal{P}_0 = \{\mathcal{A}\}$ .
2 Initialize polytope weights  $w_1(p) = \lambda(p), p \in \mathcal{P}_0$ .
3 Tune learning and exploration rates  $\eta = \gamma \leq 1/2$ , as specified in the analysis.
4 for  $t \leftarrow 1$  to  $T$  do
5   Compute  $\forall p \in \mathcal{P}_t : \pi_t(p) = (1 - \gamma)q_t(p) + \gamma \frac{\lambda(p)}{\lambda(\mathcal{A})}$ . // distribution over polytopes
   /* Two-stage sampling: first, polytope, second, draw action from within. */
6   Select polytope  $p_t \sim \pi_t$  from which you draw action  $\alpha_t \sim \text{Unif}(p_t)$  and commit to  $\alpha_t$ .
7   Observe the agent's response  $(\mathbf{r}_t(\alpha_t), y_t)$  to committed  $\alpha_t$ .
   /* Space partitioning into smaller polytopes.  $\mathcal{P}_t$ : current polytopes set. */
8   Define a new set of polytopes  $\mathcal{P}_{t+1} = \mathcal{P}_{t+1}^u(\alpha_t) \cup \mathcal{P}_{t+1}^m(\alpha_t) \cup \mathcal{P}_{t+1}^l(\alpha_t)$ , where:
9   for each polytope  $p \in \mathcal{P}_t$  do
10    Add in  $\mathcal{P}_{t+1}^u(\alpha_t)$  the non-empty intersection  $p \cap H^+(\beta_t^u(\alpha_t))$  // upper polytopes set
11    Add in  $\mathcal{P}_{t+1}^l(\alpha_t)$  the non-empty intersection  $p \cap H^-(\beta_t^l(\alpha_t))$  // lower polytopes set
12    Add in  $\mathcal{P}_{t+1}^m(\alpha_t)$  the non-empty remainder of  $p$ . // middle polytopes set
13   Compute  $\hat{\ell}(\alpha_t, \mathbf{r}_t(\alpha_t), y_t) = \frac{\ell(\alpha_t, \mathbf{r}_t(\alpha_t), y_t)}{\mathbb{P}_t^{\text{in}}[\alpha_t]}$ . // loss estimator for chosen action
14   for each polytope  $p \in \mathcal{P}_{t+1}$  do
    /* upper and lower polytopes get full information */
15    Compute  $\hat{\ell}(p, \mathbf{r}_t(p), y_t) = \frac{\ell(p, \mathbf{r}_t(p), y_t) \cdot \mathbb{1}\{p \subseteq \mathcal{P}_{t+1}^u(\alpha_t) \cup \mathcal{P}_{t+1}^l(\alpha_t)\}}{\mathbb{P}_t^{\text{in}}[p]}$ .
    /* weight scaling with the Lebesgue measure of the polytope */
16    Update  $w_{t+1}(p) = \lambda(p) \exp\left(-\eta \sum_{\tau=1}^t \hat{\ell}(p, \mathbf{r}_\tau(p), y_\tau)\right)$ ,  $q_{t+1}(p) = \frac{w_{t+1}(p)}{\sum_{p' \in \mathcal{P}_{t+1}} w_{t+1}(p')}$ .
```

Main result II

An algorithm that

- learns to adaptively partition the learner's action space according to the agent's responses, and
- minimizes the Stackelberg regret for which we prove nearly matching upper and lower regret bounds.

Theorem 4.1. *Given a finite horizon T the Stackelberg regret incurred by GRINDER (Algo. 2) is:*

$$\mathcal{R}(T) \leq \mathcal{O} \left(\sqrt{T \cdot \log \left(T \cdot \frac{\lambda(\mathcal{A})}{\lambda(\underline{p}_\delta)} \right) \cdot \log \left(\frac{\lambda(\mathcal{A})}{\lambda(\underline{p}_\delta)} \right)} \right)$$

where by $\lambda(A)$ we denote the Lebesgue measure of any measurable space A , and by $\underline{p}_\delta$ we denote the polytope with the smallest Lebesgue measure that is induced by GRINDER after T rounds' partition.

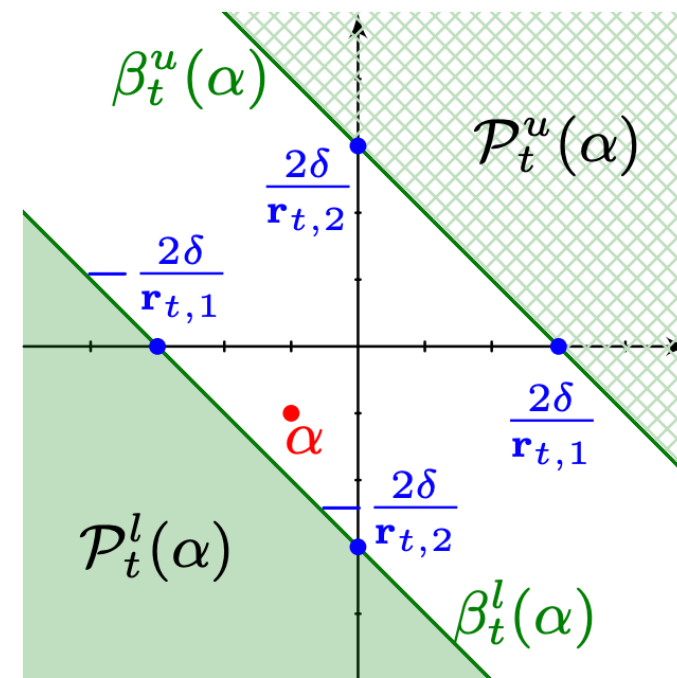


Figure 3: Polytope partitioning for $d = 2$. $\mathbf{r}_{t,1}, \mathbf{r}_{t,2}$ correspond to the x_1 and x_2 coordinates of $\mathbf{r}_t(\alpha)$.

Strategy-proof Learning with Advice

Eric Balkanski, Cherlin Zhu

as in prediction 😊

Senem Işık

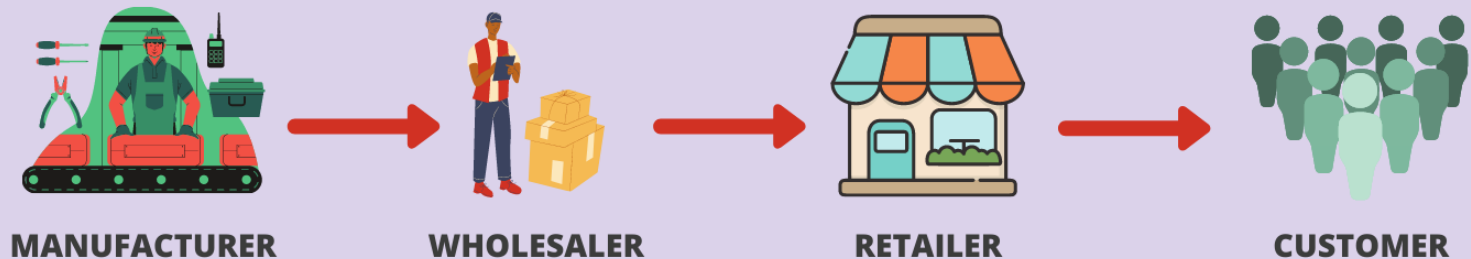
Motivating Example

Retail distribution to optimize inventory shipping.

- For **Zara**, part of the training data is provided by store managers who report to the central warehouses.
- Since managers have financial incentives to increase sales in their own stores, they may **manipulate** the predicted demand, especially there is a limited supply of a high-selling item.

RETAILDOGMA

RETAIL BUSINESS MODEL



The strategic learning model

Each agent $i \in \{1, \dots, n\}$ has **public datapoints** $\{x_{i,j}\}_j$ and **private labels** $\{y_{i,j}\}_j$ and reports a **set** $S_i = \{(x_{i,j}, y_{i,j}^*)\}_j$ of labeled data points to the learner.

The goal of the mechanism designer is to learn a function $f^* \in \mathcal{F}$ (where $\mathcal{F}: \mathcal{X} \rightarrow \mathcal{Y}$ is a hypothesis class over which we are searching) that minimizes **global** risk

$$f^* = \arg \min_{f \in \mathcal{F}} \sum_{i=1}^n \sum_{j=1}^{|S_i|} \ell(f(x_{i,j}), y_{i,j})$$

Agents are **strategic** and aim to minimize their **personal** risk

$$y_i^* = \arg \min_{\tilde{y}_i} \sum_{j=1}^{|S_i|} \ell(f^*(x_{i,j}), y_{i,j})$$

where f^* depends on y_i^* .

The strategic learning model

Each agent $i \in \{1, \dots, n\}$ has **public datapoints** $\{x_{i,j}\}_j$ and **private labels** $\{y_{i,j}\}_j$ and reports a **set** $S_i = \{(x_{i,j}, y_{i,j}^*)\}_j$ of labeled data points to the learner.

The goal of the mechanism designer is to learn a function $f^* \in \mathcal{F}$ (where $\mathcal{F}: \mathcal{X} \rightarrow \mathcal{Y}$ is a hypothesis class over which we are searching) that minimizes **global** risk

$$f^* = \arg \min_{f \in \mathcal{F}} \sum_{i=1}^n \sum_{j=1}^{|S_i|} \ell(f(x_{i,j}), y_{i,j})$$

Agents are **strategic** and aim to minimize their **personal** risk

$$y_i^* = \arg \min_{\tilde{y}_i} \sum_{j=1}^{|S_i|} \ell(f^*(x_{i,j}), y_{i,j})$$

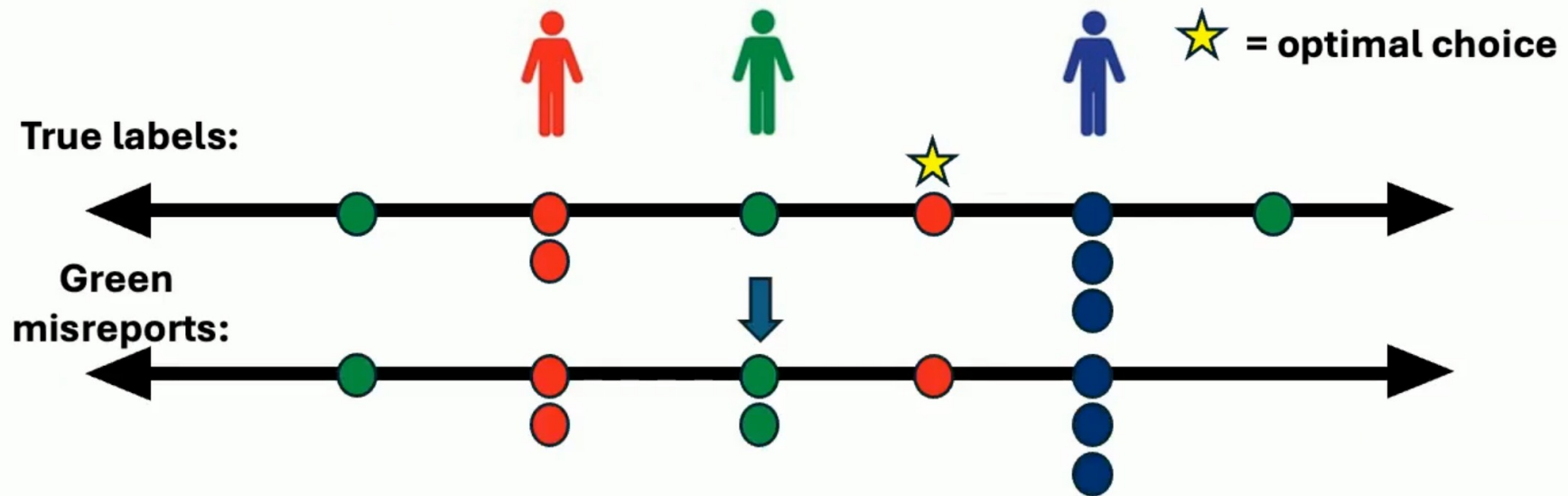
where f^* depends on y_i^* .

where f^* is **strategy-proof**: no agent can decrease their personal risk by misreporting meaning we have (i.e. $y_i^* = y_i$).

Learning constant functions

Optimal functions outputs median over all $y_{i,j}$'s but this is **not** strategy-proof!
(Dekel, Fischer, Procaccia'10)

- Requiring strategy-proofness comes at the cost of optimality!



Real-life is not as pessimistic

Input: dataset $\mathcal{S}_i = \{(x_{i,j}, y_{i,j}^*)\}_j$
+ advice $\tilde{f}(x) = \tilde{a}$
(this is **weaker** than real y)

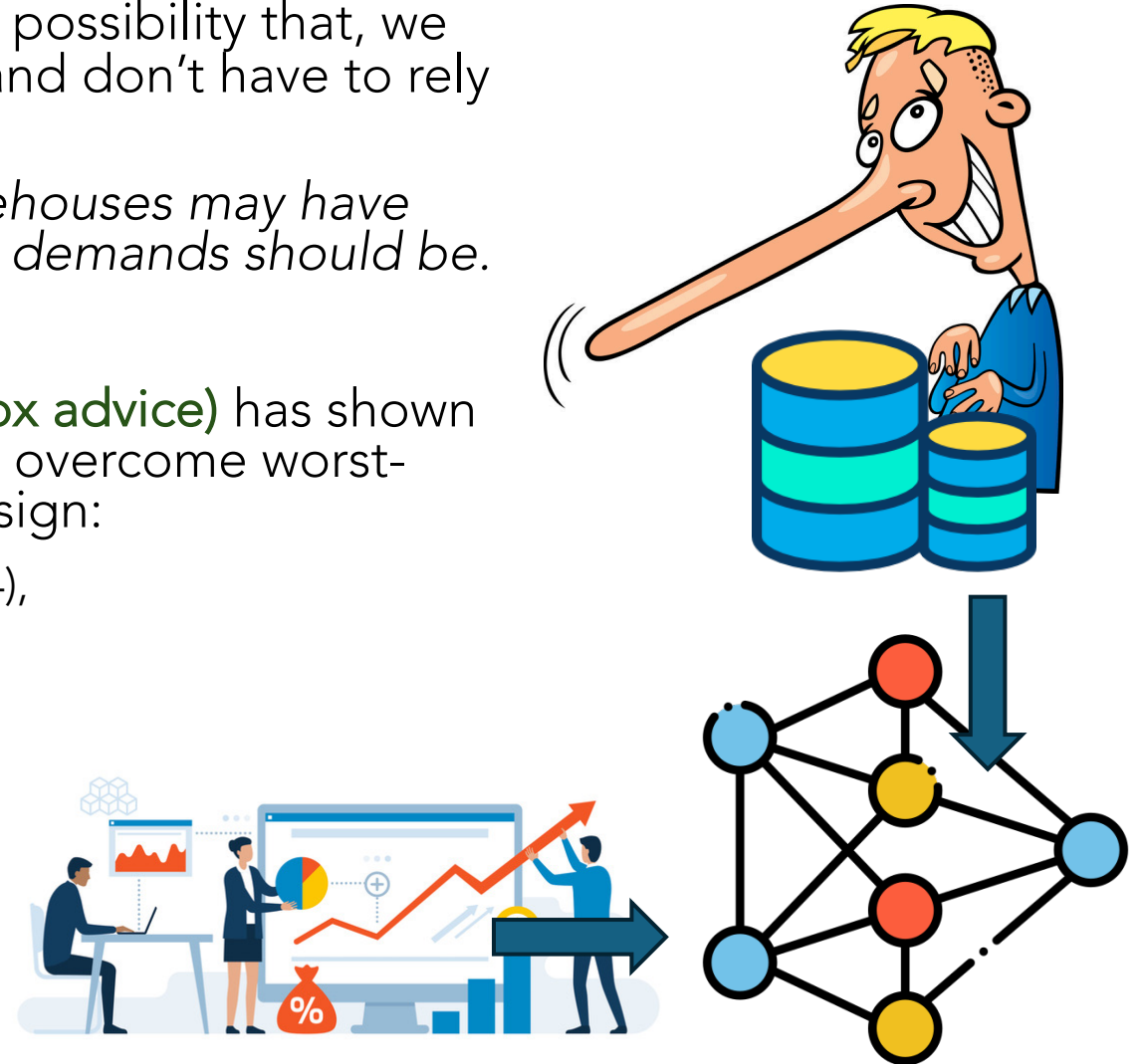
Strategic classification viewpoint overlooks the possibility that, we might have some **prediction** on the user data and don't have to rely on the user **only**.

eg. *it is reasonable to assume that central warehouses may have access to rough estimates of what these future demands should be.*

Related work (mechanism design with black-box advice) has shown that such side information can be leveraged to overcome worst-case bounds in (strategy-proof) mechanism design:

- facility location (Agrawal et al, 2022; Barak et al, 2024),
- auction design (Balkanski et al, 2024; Xu et al 2022),
- strategic scheduling (Balkanski et al, 2023),
- strategic assignment (Colini-Baldeschi, 2024),
- metric distortion (Berger et al, 2024)

...



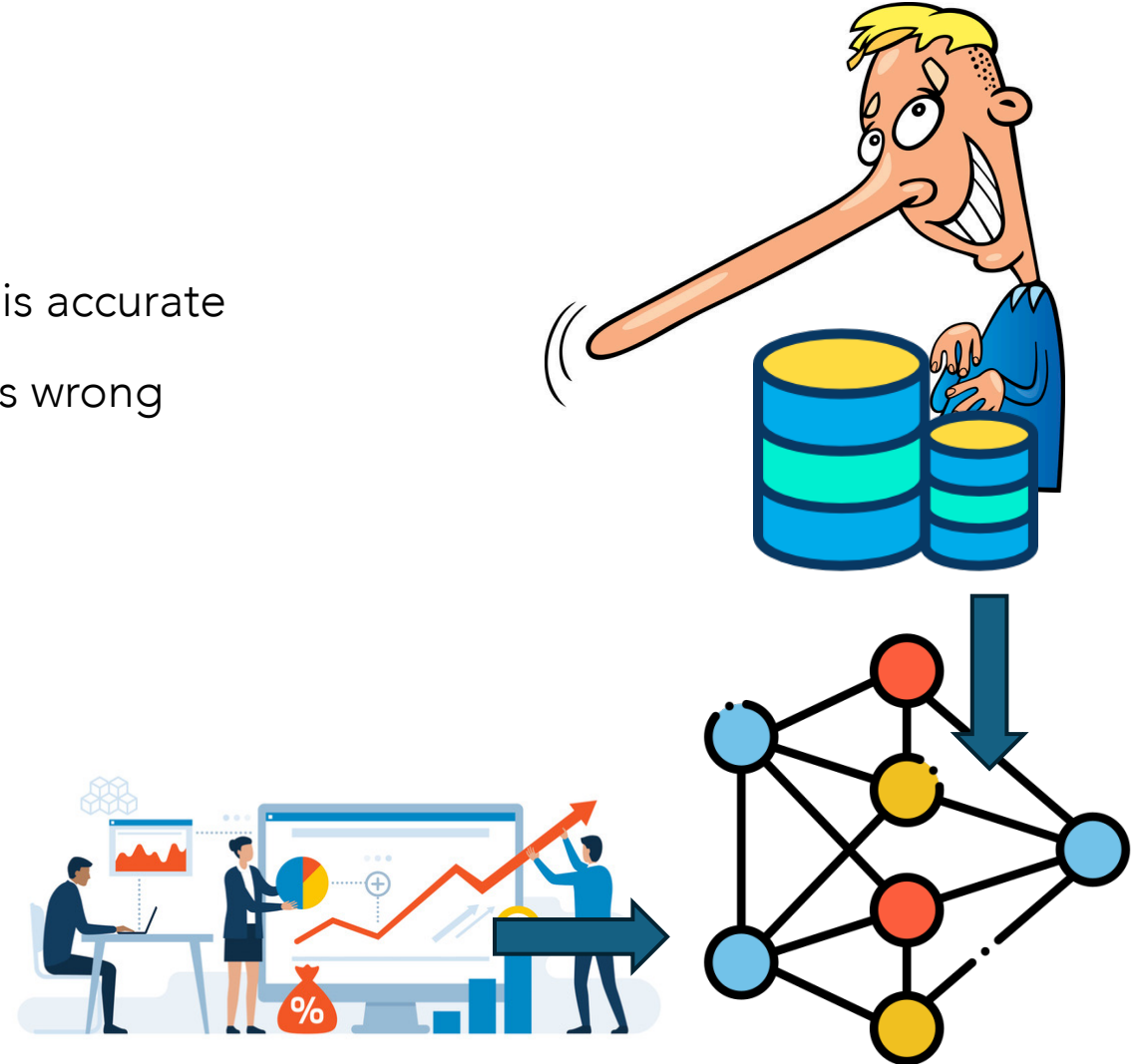
Real-life is not as pessimistic

Input: dataset $\mathcal{S}_i = \{(x_{i,j}, y_{i,j}^*)\}_j$
+ advice $\tilde{f}(x) = \tilde{a}$
(this is **weaker** than real y)

Mechanism design with black-box advice

- **Consistency:** approximation ratio when advice is accurate
- **Robustness:** approximation ratio when advice is wrong

This is a trade-off for algorithms,
giving a pareto frontier.



Main results I

Without advice, there is deterministic strategy proof mechanism that achieves **3-approximation**, and this is tight for deterministic strategy proof mechanisms (Dekel, Fischer, Procaccia'10).

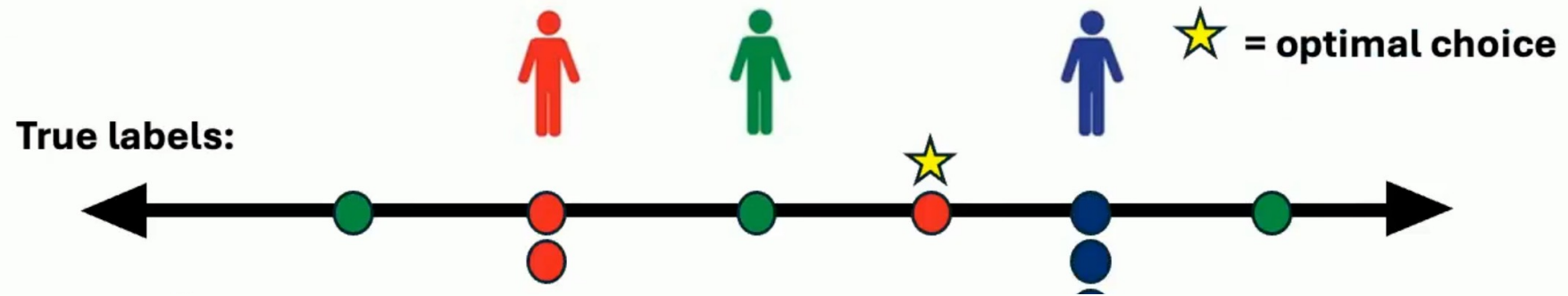
Theorem. *For any $\mathbb{Y} \subseteq \mathbb{R}$ and $\gamma \in (0, 2]$, there is a deterministic and strategyproof mechanism for the class of constant functions $\mathcal{F}_{\mathbb{Y}} = \{f_a(x) = a : a \in \mathbb{Y}\}$ that is $1 + \gamma$ consistent and $1 + 4/\gamma$ robust. Moreover, for any T sufficiently large and $\gamma \in (0, 2]$, any deterministic, strategyproof, and $1 + \gamma$ consistent mechanism for the class of constant functions $\mathcal{F}_{\mathbb{R}_{\leq T}}$ is $1 + 4/\gamma - \Omega(1/n)$ robust.*

- When $\gamma = 2$, recovers the 3-approximation of previous work and $\gamma = \varepsilon$, gets near-optimal consistency and constant robustness.
- Their algorithm is **group-strategy-proof**.
- *Presents approximation as function of advice error.*

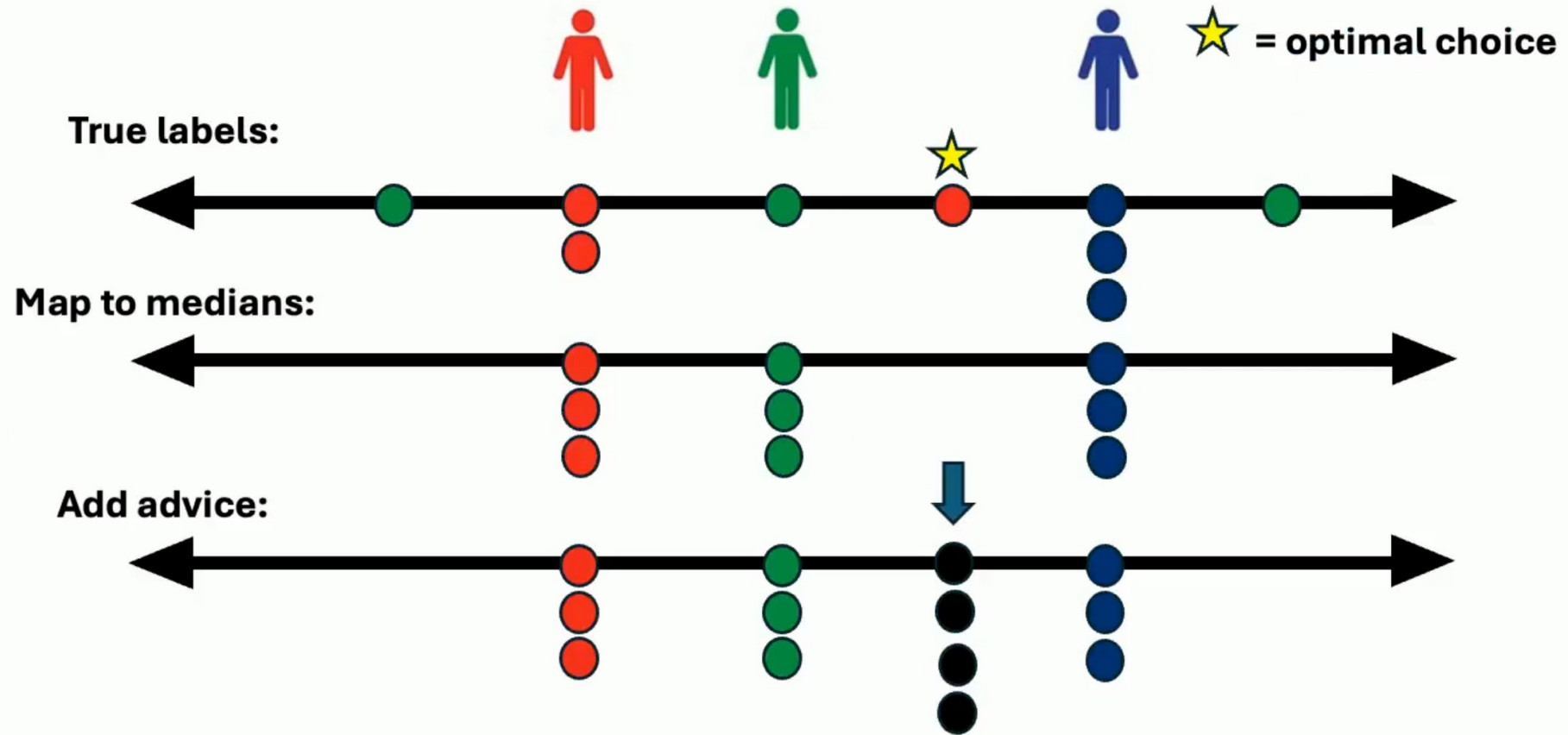
The Mechanism: Constant Fit with Advice

- (1) Set parameter $\gamma = [0,2]$
- (2) Input: dataset $S_i = \{(x_{i,j}, y_{i,j}^*)\}_j$ + advice $\tilde{f}(x) = \tilde{a}$ (this is weaker than real y)
- (3) For each agent i , select the **median** m_i of $\{y_{i,j}^*\}_j$.
- (4) Create a new dataset $S' = \bigcup_i \{m_i\}_{j \in [|S_i|]} \cup \{\tilde{a}\}_{k \in \gamma|S|}$
- (5) Return median point of S' .

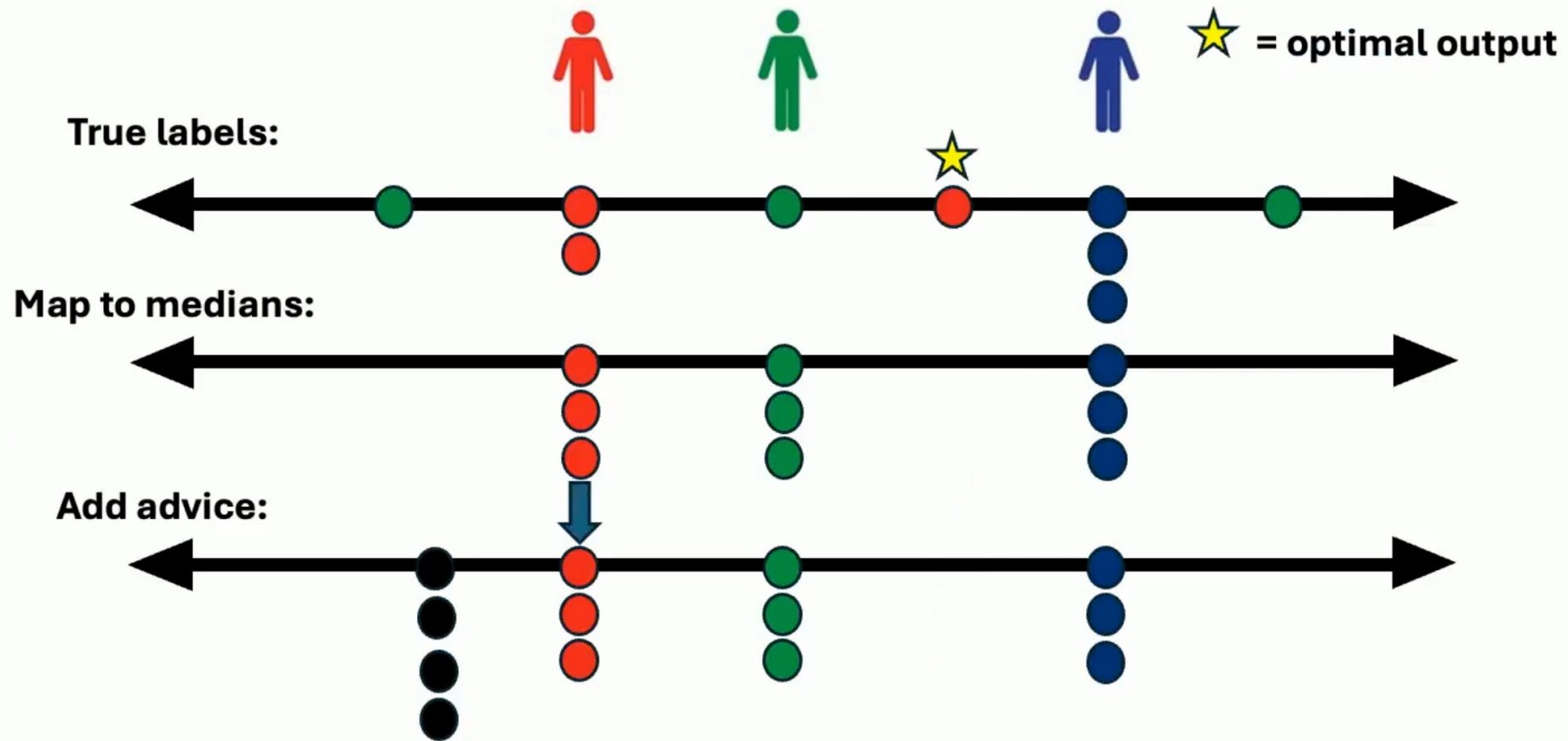
The Mechanism in action



The Mechanism in action

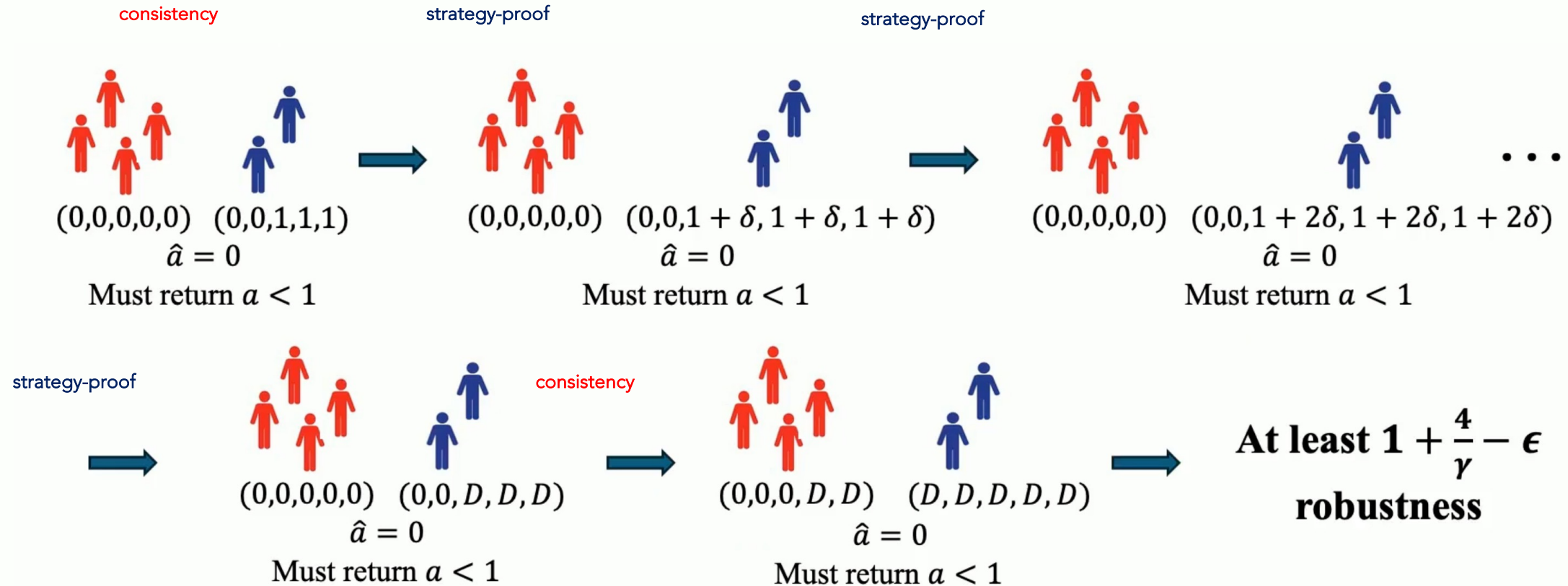


The Mechanism in action



Incompatibility: *consistency vs robustness*

Requiring strategy-proofness + $(1 + \gamma)$ -consistency restricts the output over a series of instances...



Results

	Function class		No advice	With advice	
				Consistency	Robustness
Regression	Constant	det.	3^*	$1 + \gamma$	$1 + \frac{4}{\gamma}$
	Homogeneous linear	det.	3^*	$1 + \gamma$	$1 + \frac{4}{\gamma}$
Classification	General labelings	det.	$O(n)^\dagger$	if $o(n)$	must be unbounded
		rand.	$3^\dagger$	$3 - \frac{2}{n}$	$\Omega(n)$
	Two labelings	det.	$3^\dagger$	$1 + \gamma$	$1 + \frac{4}{\gamma}$
		rand.	$2^\dagger$	$1 + \gamma$	$1 + \frac{1}{\gamma}$

Table 1: Summary of results. The results are for $\gamma \in (0, 2]$ except random classification over two labelings, which is for $\gamma \in (0, 1]$. Results labeled with $\star$ are from Dekel et al. [15], and those labeled with $\dagger$ are from Meir et al. [31].

Advice is not as useful for classification

Positive results generalize to the setting where each agent's data **is sampled from a distribution** using techniques from PAC-learning.

Question I

Each agent $i \in \{1, \dots, n\}$ has **private datapoints** $\{x_{i,j}\}_j$ and **private labels** $\{y_{i,j}\}_j$ and reports a **set** $S_i = \{(\hat{x}_{i,j}, \hat{y}_{i,j})\}_j$ of **private** data points to the learner

(possibly there is a **budget** on how much agents can manipulate too eg. **δ -Bounded agents**)

How would having a **prediction for f^*** help in the **offline setting**?

- (1) Nature draws $x \sim \mathcal{D}$.
- (2) The principal commits to a decision-making rule $f: \mathcal{X} \rightarrow \mathcal{Y}$.
- (3) The agents observe f and their point (x, y) .
- (4) Given f, x, y the agents choose $\hat{x} = \hat{x}(f; x, y) \in \mathcal{X}$ is the best-response of the agent (given pair (x, y)) to the principal's rule f .
- (5) The agent reports point $\hat{x}$ to the principal. **But the principal also has access to an independent prediction.**

Find a function $f^* \in \mathcal{F}$ such that

$$f^* = \arg \min_{f \in \mathcal{F}} \mathbb{E}_{x \in \mathcal{D}} [\ell(h^*(x), f(\hat{x}(f)))]$$

What robustness/consistency approximation we can get for the following metric?

$$\mathbb{E}_{x \in \mathcal{D}} [\ell(h^*(x), f(\hat{x}(f)))]$$

Question I continued

Each agent $i \in \{1, \dots, n\}$ has **private datapoints** $\{x_{i,j}\}_j$ and **private labels** $\{y_{i,j}\}_j$ and reports a **set** $S_i = \{(\hat{x}_{i,j}, \hat{y}_{i,j})\}_j$

+ What if there is a **budget** on how much agents can manipulate too eg. **δ -Bounded agents**.

→ We can just worry about **strategy proof mechanism** against **bounded agents** → could help with **robustness!**

First Step: a sequence of counterexamples

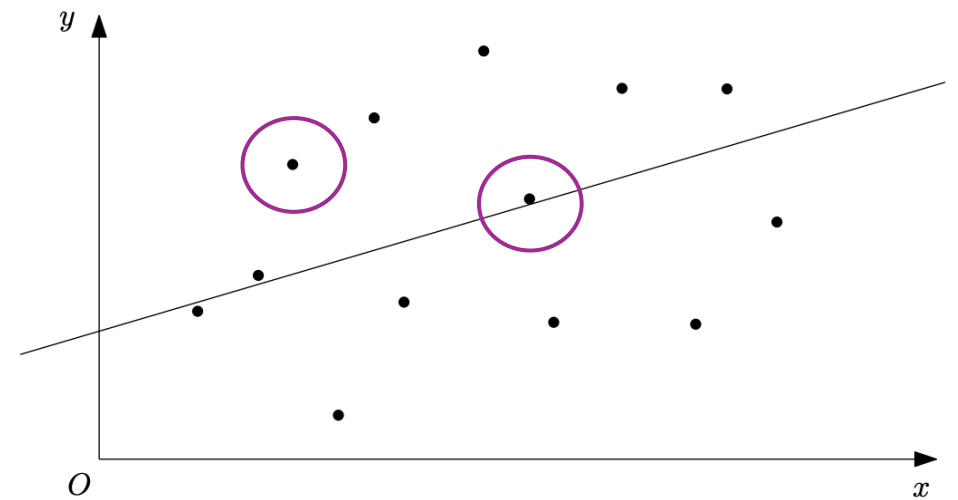
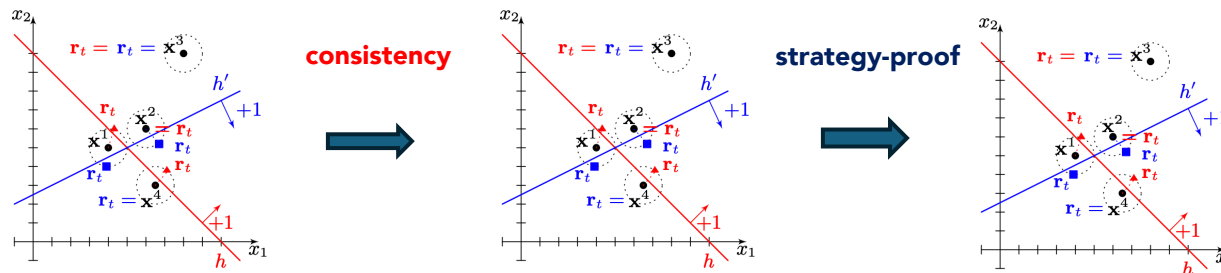


Figure 1: A set of points and the “best-fit line.”



Question II

Each agent $i \in \{1, \dots, n\}$ has (private) datapoints $\{x_{i,j}\}_j$ and private labels $\{y_{i,j}\}_j$ and reports a set $S_i = \{(\hat{x}_{i,j}, \hat{y}_{i,j})\}_j$ of private data points to the learner

(possibly there is a budget on how much agents can manipulate too eg. δ -Bounded agents)

How would having a prediction for f^* help in the offline setting?

- (1) Nature draws $x \sim \mathcal{D}$.
- (2) The principal commits to a decision-making rule $f: \mathcal{X} \rightarrow \mathcal{Y}$.
- (3) The agents observe f and their point (x, y) . **What if the agent knows that what prediction the principal has?**
- (4) Given f, x, y the agents choose $\hat{x} = \hat{x}(f; x, y) \in \mathcal{X}$ is the best-response of the agent (given pair (x, y)) to the principal's rule f .
- (5) The agent reports point $\hat{x}$ to the principal. **But the principal also has access to an independent prediction.**

Find a function $f^* \in \mathcal{F}$ such that

$$f^* = \arg \min_{f \in \mathcal{F}} \mathbb{E}_{x \in \mathcal{D}} [\ell(h^*(x), f(\hat{x}(f)))]$$

Thank you!